MODULE 1 — Process & Thread

Senior SWE Interview Track — Operating Systems

1.1 Process vs Thread

1. Why Interviewers Ask This

It is the #1 OS screening question at Google, Meta, and Amazon because your answer reveals whether you understand isolation, memory layout, and the cost model of concurrency — the foundation for every follow-up on scaling backend services.

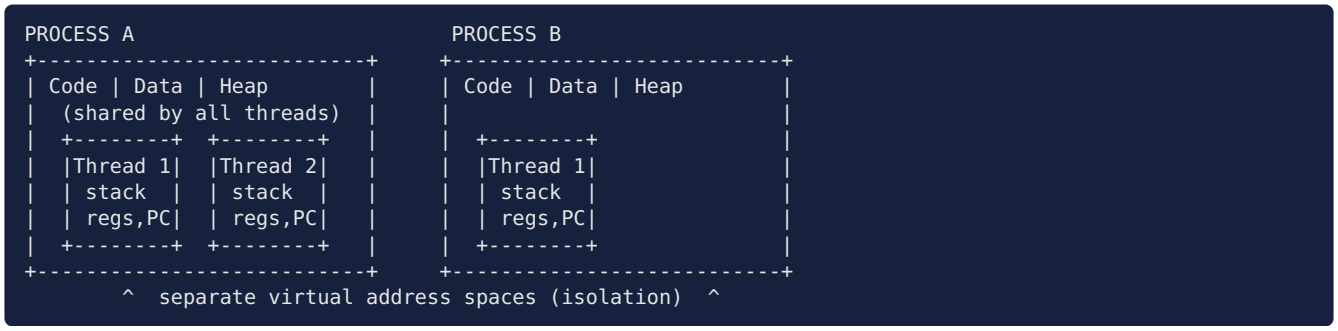
2. Core Concept

- A **process** is an instance of a running program: its own virtual address space, file descriptor table, credentials, and at least one thread.
- A **thread** is an execution context (registers + stack + program counter) *inside* a process. All threads in a process share the address space, heap, globals, and open file descriptors; each has its own stack and register state.
- Isolation boundary = process. Concurrency unit = thread.

3. Internal Working

- On Linux both are created via `clone()`. A process (`fork()`) gets a new `mm_struct` (address space); a thread (`pthread_create`) passes `CLONE_VM | CLONE_FILES | CLONE_SIGHAND | CLONE_THREAD`, so the kernel reuses the parent's address space and FD table.
- The kernel schedules **tasks** (`task_struct`) — it barely distinguishes process vs thread; the difference is what is *shared*.
- Crash semantics: a segfault in one thread kills the whole process (shared address space); a crashed child process leaves siblings alive.

4. ASCII Diagram



5. Real Production Example

- **Chrome**: one process per tab/site → a crashing tab or exploited renderer can't touch other tabs (isolation over memory cost).
- **Nginx**: multi-process workers (no shared-state locking bugs, per-core scaling).
- **JVM services (e.g., LinkedIn, Netflix backends)**: one process, hundreds of threads sharing heap — cheap data sharing, but must synchronize.

6. Advantages

- Threads: cheap creation, fast context switch (no address-space switch), zero-copy data sharing.
- Processes: fault isolation, security isolation, independent lifecycle, no data races by default.

7. Trade-offs

- Threads: one bad pointer corrupts everyone; locking complexity; harder debugging.
- Processes: IPC is slower (copies/syscalls), higher memory footprint, slower spawn.

8. Common Mistakes

- Saying “threads are always faster” — ignores lock contention and cache-line bouncing.
- Claiming threads share stacks (they don’t — stacks are per-thread).
- Forgetting FDs are shared across threads but *copied* across `fork()`.

9. Performance Implications

- Thread context switch ≈ 1–2 µs; process switch adds TLB flush cost (unless ASID/PCID tagged).
- Thread creation ≈ tens of µs; process fork of large process is dominated by page-table copy (mitigated by COW).
- Shared heap = cache coherence traffic under contention (false sharing).

10. Common Interview Questions

- “What exactly is shared between threads and what is private?”
- “Why did Chrome pick processes per tab despite the memory cost?”
- “What happens to threads when a process forks?” (Only the calling thread exists in the child — classic trap.)

11. Follow-up Questions

- “How does the kernel represent threads vs processes?” → `task_struct` + shared `mm_struct`.
- “How would you share memory between processes?” → `shm/mmap(MAP_SHARED)`, discussed in Module 9.
- “What is a thread pool and why not spawn a thread per request?”

12. Coding/Debugging Scenarios

- Multithreaded server occasionally exits entirely: one worker thread segfaulted → whole process died. Fix: isolate risky work in child processes or harden the code.
- After `fork()` in a threaded program, child deadlocks: a lock was held by a thread that doesn’t exist in the child. Fix: `fork()` then immediately `exec()`, or `pthread_atfork`.

13. Best Practices

- Use processes for isolation boundaries (plugins, tenants, sandboxes); threads for parallelism within one trust domain.
- Prefer thread pools over unbounded thread creation.
- In threaded programs, only `async-signal-safe` calls between `fork()` and `exec()`.

14. Practice Questions

1. Design a browser architecture: what runs in which process and why?
2. A 16-core box runs a CPU-bound job — how many processes/threads and why?
3. Explain why `fork()` in a 200-thread JVM is dangerous.

1.2 Process Control Block (PCB)

1. Why Interviewers Ask This

Tests whether you know what the kernel must save/restore — the direct input to explaining context-switch cost.

2. Core Concept

The PCB is the kernel’s per-process data structure holding everything needed to stop and later resume a process. On Linux this is `task_struct`.

3. Internal Working

Key fields: - **Identification**: PID, PPID, UID/GID, process group, session. - **CPU state**: saved registers, program counter, stack pointer (in kernel stack / `pt_regs`). - **Scheduling**: state (running/ready/blocked), priority/nice, vruntime (CFS/EEVDF), CPU affinity. - **Memory**: pointer to `mm_struct` → page table root (CR3 value), VMA list. - **I/O & files**: FD table pointer, current working directory, root. - **Signals**: pending set, handler table. - **Accounting**: CPU time, limits (rlimits, cgroup membership). PCBs live on kernel run queues / wait queues; scheduler picks the next PCB and restores its state.

4. ASCII Diagram

```
task_struct (PCB)
+--------------------------+
| pid, ppid, uid           |
| state: RUNNING/READY/...  |
| sched: prio, vruntime     |
| *mm  ---> page tables     |
| *files -> fd table        |
| signals: pending, handlers|
| CPU ctx: regs, PC, SP     |
| accounting, cgroups       |
+--------------------------+
   linked into: run queue | wait queue | parent's child list
```

5. Real Production Example

`ps`, `top`, and `/proc/<pid>/*` are windows into PCBs. Kubernetes kills a container by signaling PIDs tracked via cgroups — all PCB-level metadata.

6. Advantages

Central, O(1)-accessible bookkeeping enables preemptive multitasking, fair scheduling, resource accounting, and clean teardown.

7. Trade-offs

Each task costs kernel memory (~few KB of `task_struct` + kernel stack, typically 8–16 KB) → hard limits on process/thread counts (`pid_max`, `threads-max`).

8. Common Mistakes

- Confusing the PCB with the user-space stack.
- Saying registers are stored “in the PCB” literally on Linux — they’re saved on the kernel stack (`pt_regs`), referenced from the task.
- Forgetting a zombie is exactly “a PCB kept around for the parent to read the exit status.”

9. Performance Implications

PCB size × task count bounds scalability (e.g., 100k threads ≈ ~1–2 GB of kernel stacks alone). Scheduler operations touch PCB fields — cache locality of run queues matters.

10. Common Interview Questions

- “What must be saved on a context switch?”
- “What remains after a process exits but before `wait()`?” (zombie = PCB shell)

11. Follow-up Questions

- “Where is the page-table pointer stored and when is it loaded?” (in `mm_struct`; loaded into CR3 on switch)
- “How does `top` know per-process CPU%?” (accounting fields exposed via `/proc`)

12. Coding/Debugging Scenarios

- `fork()` returns `EAGAIN` under load → hit `pid_max`/`threads-max` or cgroup pids limit; inspect `/proc/sys/kernel/pid_max`.

13. Best Practices

- Cap concurrency (pools) so task counts stay bounded; monitor `/proc` metrics rather than guessing.

14. Practice Questions

1. Walk through every PCB field the kernel touches when a blocked process becomes runnable and is scheduled.
2. Why do zombies consume almost no memory yet can still exhaust the system?

1.3 Thread Control Block (TCB)

1. Why Interviewers Ask This

Follow-up to PCB: do you understand *which* state is per-thread — the key to explaining why thread switches are cheaper.

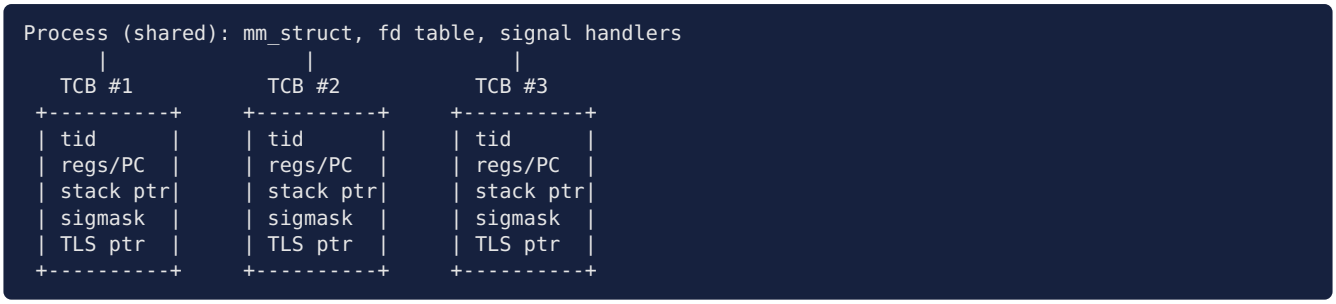
2. Core Concept

The TCB stores per-thread state: thread ID, register set, program counter, stack pointer, scheduling state/priority, signal mask, and thread-local storage (TLS) pointer. Shared process resources (address space, FDs) are referenced, not owned.

3. Internal Working

- Linux: each thread is a `task_struct` sharing `mm`, `files`, `sighand` with siblings; `tgid` = process ID, `pid` = thread ID.
- User-level threading libraries (Go goroutines, Java virtual threads) keep their own tiny TCBs in user space and multiplex over kernel threads.
- TLS is reached via a dedicated register (`FS`/`GS` on x86-64) pointing at the thread's TLS block.

4. ASCII Diagram



5. Real Production Example

Go runtime `g` structs (goroutine TCBs) are ~2–4 KB starting stacks → millions of goroutines per process, versus ~8 MB default pthread stacks.

6. Advantages

Minimal per-thread state → fast switch, cheap creation, natural data sharing.

7. Trade-offs

No isolation; per-thread stacks still consume virtual memory; signal handling across threads is subtle (handlers shared, masks per-thread).

8. Common Mistakes

- Assuming each thread has its own heap.
- Assuming signal *handlers* are per-thread (only the *mask* is).

9. Performance Implications

Default 8 MB stacks × 10k threads = 80 GB *virtual* (fine) but resident touch grows real memory; tune `pthread_attr_setstacksize` for massive thread counts.

10. Common Interview Questions

- "What's in a TCB vs PCB?"
- "Why can Go run 1M goroutines but pthreads can't do 1M threads comfortably?"

11. Follow-up Questions

- "How does `errno` work with threads?" (TLS variable)
- "Which thread receives a `SIGTERM` sent to the process?" (any thread with it unblocked)

12. Coding/Debugging Scenarios

- Stack overflow in a deep-recursion worker thread → crashes with SIGSEGV near guard page; fix by larger stack or iterative algorithm.

13. Best Practices

- Size stacks deliberately for high-thread-count services; use TLS instead of global mutable state.

14. Practice Questions

1. Compare memory cost of 100k pthreads vs 100k goroutines.
2. Explain how a user-level scheduler switches between two TCBs without entering the kernel.

1.4 User Threads vs Kernel Threads

1. Why Interviewers Ask This

Directly relevant to modern runtimes (Go, Java virtual threads, async runtimes). Interviewers use it to test whether you understand *who schedules what* and the blocking-syscall trap.

2. Core Concept

- **Kernel thread**: known to and scheduled by the kernel (1 `task_struct` each). Can run in parallel on multiple cores; blocking syscalls block only that thread.
- **User thread (green thread)**: scheduled by a user-space runtime; the kernel sees only the carrier kernel threads. Switches are pure user-space function calls.

3. Internal Working

- User-level switch: save callee-saved registers + SP, swap to another user stack — no syscall, ~tens of ns.
- Kernel-level switch: syscall/interrupt into kernel, scheduler runs, restores another task — ~1–2 µs.
- The classic failure: a user thread makes a blocking syscall → the carrier kernel thread blocks → *all* user threads on it stall. Runtimes fix this with non-blocking I/O + event loop (Go netpoller) or by handing off blocked carriers (Go `sysmon`, Java virtual thread unmount).

4. ASCII Diagram

```
1:1 model (pthreads)            M:N model (Go, Java virtual threads)
UserT1 UserT2 UserT3            g1 g2 g3 g4 g5 g6   (user threads)
  |      |      |                 \  |  /   \ | /
  K1     K2     K3                   M1        M2     (kernel threads)
  |      |      |                    \        /
 CPU    CPU    CPU                     CPU  CPU
Kernel schedules all          Runtime schedules g's onto M's;
                              kernel schedules M's onto CPUs
```

5. Real Production Example

- Go: goroutines (M:N, GMP scheduler) — powers high-concurrency services at Uber, Cloudflare.
- Java 21 virtual threads: M:N mounted on carrier platform threads.
- Node.js: single kernel thread + event loop (cooperative, callback-based).

6. Advantages

- User threads: ~100× cheaper switch/creation, millions of concurrent tasks, custom scheduling.
- Kernel threads: true parallelism, kernel handles blocking, preemption for free.

7. Trade-offs

- User threads: blocking-syscall hazard, can't be preempted by kernel (runtime must insert preemption points), poor fit for CPU-bound work without M:N.
- Kernel threads: heavier memory and switch cost; ~10k practical ceiling for many workloads.

8. Common Mistakes

- "Green threads run in parallel" on a single carrier — they don't; only M:N with multiple carriers gives parallelism.
- Ignoring that file I/O on Linux is effectively always blocking (no epoll for regular files) — Go compensates with more carrier threads.

9. Performance Implications

M:N wins for I/O-bound high-concurrency (chat servers, proxies). 1:1 is simpler and fine up to thousands of threads. Context-switch overhead at 100k+ concurrent connections makes 1:1 infeasible → C10K/C10M problem.

10. Common Interview Questions

- "How does Go run a million goroutines on 8 cores?"
- "What happens when a goroutine calls a blocking syscall?"

11. Follow-up Questions

- "How does the runtime preempt a tight loop with no function calls?" (Go: async preemption via signals since 1.14)
- "Compare virtual threads to an async/await event loop."

12. Coding/Debugging Scenarios

- Latency spikes in a Go service: goroutines pile up behind a blocking cgo/syscall hogging carrier threads → check `runtime` metrics, move blocking work to a bounded pool.

13. Best Practices

- I/O-bound massive concurrency → green/virtual threads or async I/O. CPU-bound → ~1 kernel thread per core.
- Never do blocking calls inside an event loop thread.

14. Practice Questions

1. Design the threading model for a websocket gateway with 2M idle connections.
2. Why does Java virtual threads' "pinning" (synchronized blocks) hurt, and how do you detect it?

1.5 Context Switching

1. Why Interviewers Ask This

Cost of context switching is the quantitative backbone of every "threads vs async" and "why is my p99 bad" discussion.

2. Core Concept

A context switch is the kernel saving the CPU state of the current task and restoring another's. Triggers: time-slice expiry, blocking (I/O, lock), higher-priority wakeup, explicit yield, interrupts.

3. Internal Working

1. Interrupt/syscall enters kernel mode.
2. Save current registers/PC/SP into the task's kernel stack (`pt_regs`).
3. Scheduler (`schedule()`) picks next task from run queue.
4. If a different process: load new page-table root (CR3 write → TLB implications; PCID mitigates full flush).
5. `switch_to`: swap kernel stacks and registers; return in the new task's context. Indirect cost dominates: TLB misses and cold CPU caches after the switch (can be tens of µs of degraded IPC).

4. ASCII Diagram

```
Task A running --> timer interrupt
   | save A: regs, PC, SP  -> A's PCB/kstack
   | scheduler: pick B
   | if B in other process: load B's CR3 (TLB!)
   | restore B: regs, PC, SP <- B's PCB/kstack
Task B running
Direct cost ~1-2us; indirect (cache/TLB refill) often >> direct
```

5. Real Production Example

A trading system or Redis pins threads to cores (`taskset`, `isolcpus`) precisely to avoid switch + cache-pollution jitter. Overloaded API servers show high `cs` in `vmstat` and rising p99 while CPU% looks "fine".

6. Advantages

Enables preemptive multitasking, fairness, and responsiveness on limited cores.

7. Trade-offs

Pure overhead — no user work done. Voluntary (blocking) switches are a signal of I/O-bound behavior; involuntary ones signal CPU contention.

8. Common Mistakes

- Quoting only the direct cost and ignoring cache/TLB pollution.
- Confusing a *mode* switch (user→kernel syscall, no task change) with a *context* switch.
- Thinking thread switches within a process flush the TLB (they don't — same address space).

9. Performance Implications

At 100k switches/sec × ~2 µs direct ≈ 20% of a core gone, before cache effects. `pidstat -w` splits voluntary vs involuntary; high involuntary = too many runnable threads per core.

10. Common Interview Questions

- "Walk me through exactly what happens during a context switch."
- "Why is switching between threads of one process cheaper?"

11. Follow-up Questions

- "What is PCID/ASID and why does it matter?" (tagged TLB entries avoid full flush)
- "How does this cost drive the design of epoll-based servers?" (fewer threads → fewer switches)

12. Coding/Debugging Scenarios

- Service p99 degrades as thread pool grows from 200→2000 threads on 16 cores: involuntary switches skyrocket. Fix: bounded pool ≈ cores × (1 + wait/compute ratio), or async I/O.

13. Best Practices

- Keep runnable-threads ≈ core count; pin latency-critical threads; measure with `vmstat`, `pidstat -w`, `perf sched`.

14. Practice Questions

1. Estimate switch overhead for 50k RPS where every request blocks twice.
2. Explain why Meltdown mitigations (KPTI) made syscalls/context switches costlier.

1.6 Process States

1. Why Interviewers Ask This

State transitions explain every `top` output and every "process is stuck in D state" incident — a favorite senior debugging probe.

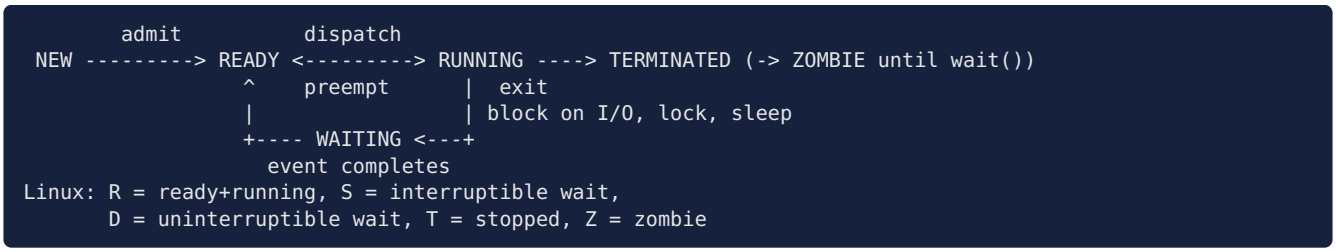
2. Core Concept

Canonical five-state model: **New → Ready → Running → Waiting(Blocked) → Terminated**, with Ready↔Running via scheduler and Running→Waiting on blocking. Linux states: `R` (running/runnable), `S` (interruptible sleep), `D` (uninterruptible sleep), `T` (stopped), `Z` (zombie).

3. Internal Working

- Ready: on a per-CPU run queue.
- Running: currently on a CPU.
- Waiting: on a wait queue tied to an event (I/O completion, lock, timer). `S` can be woken by signals; `D` cannot (usually mid-I/O in the kernel — unkillable).
- Zombie: exited; PCB retained until parent `wait()`s (Module 9).

4. ASCII Diagram



5. Real Production Example

NFS server outage → processes pile up in `D` state, load average climbs into the hundreds while CPU is idle (load counts R **and** D on Linux). `kill -9` does nothing — classic on-call scenario.

6. Advantages

Explicit states let the scheduler skip blocked tasks and let operators reason about system health.

7. Trade-offs

`D` state is required for data integrity mid-I/O but creates unkillable processes; `TASK_KILLABLE` is the kernel's compromise.

8. Common Mistakes

- Believing `kill -9` kills anything ("`D` state and zombies are immune — zombies are already dead").
- Reading Linux load average as CPU-only.

9. Performance Implications

Many `D`-state tasks = storage/network-storage bottleneck, not CPU. Ready-queue length per core >> 1 sustained = CPU saturation.

10. Common Interview Questions

- "Draw the process state diagram."
- "Load average is 80 on an 8-core box but CPU is 10% — explain." (D-state pileup)

11. Follow-up Questions

- "Difference between S and D sleep?" "Why can't you kill a D-state process?"

12. Coding/Debugging Scenarios

- `ps aux | awk '$8 ~ /D/'` to find D-state tasks; `cat /proc/<pid>/stack` shows they're stuck in a filesystem/driver path → fix the storage layer.

13. Best Practices

- Use timeouts on I/O (soft NFS mounts, socket timeouts) so tasks sleep interruptibly; alert on D-state counts.

14. Practice Questions

1. Trace all state transitions for one HTTP request handled by a blocking server.
2. Why does a zombie stay in `Z` even after `kill -9`?

1.7 Thread Lifecycle

1. Why Interviewers Ask This

Java/backend interviews love it (`NEW/RUNNABLE/BLOCKED/WAITING/TIMED_WAITING/TERMINATED`), and it maps to thread-dump reading — a real production skill.

2. Core Concept

A thread goes: created → runnable → running → (blocked on lock | waiting on condition | sleeping) → terminated → joined/detached. Java's states in thread dumps: `BLOCKED` (waiting for a monitor), `WAITING` (`wait()`/`park()`), `TIMED_WAITING` (`sleep`, timed wait).

3. Internal Working

- `pthread_create` → `clone()` → new task in ready state.
- Blocking on a mutex parks the thread on a futex wait queue (kernel), state = sleeping.
- On exit, resources aren't freed until `pthread_join` (or the thread is detached) — the thread equivalent of zombie.

4. ASCII Diagram

```
NEW --start()--> RUNNABLE <--scheduler--> RUNNING --run ends--> TERMINATED --join()
                    ^  ^  ^                  |  |  |
    lock acquired --+  |  +-- notify/signal |  |  +-- sleep(t)/timed wait
                       |                    |  +----> WAITING  (wait/park)
                       +----- BLOCKED  <----+  (contended lock)
                                      (TIMED_WAITING for sleep/timed ops)
```

5. Real Production Example

Diagnosing a frozen Java service: `jstack` dump shows 200 threads `BLOCKED` on one monitor held by a thread doing a slow DB call — lifecycle states are the vocabulary of that diagnosis.

6. Advantages

Well-defined states → thread dumps, profilers, and deadlock detectors work.

7. Trade-offs

Unjoined (non-detached) finished threads leak stack memory; too many threads in `RUNNABLE` = CPU thrash; in `BLOCKED` = contention.

8. Common Mistakes

- Confusing `BLOCKED` (lock) with `WAITING` (condition) in Java dumps.
- Forgetting to join/detach → resource leak.
- Assuming `RUNNABLE` means "on CPU" (it includes ready-and-waiting; and in Java, even threads in native I/O).

9. Performance Implications

Thread-dump state histograms are the fastest triage: mostly BLOCKED → lock contention; mostly WAITING on pool queue → underutilized or upstream-starved; mostly RUNNABLE → CPU-bound.

10. Common Interview Questions

- "Explain Java thread states and what each means in a thread dump."
- "Difference between `sleep()` and `wait()`?" (wait releases the monitor; sleep doesn't)

11. Follow-up Questions

- "What does `park()`/`unpark()` do vs `wait()`/`notify()`?"
- "How do you find which thread holds the lock others are blocked on?" (dump shows monitor owner)

12. Coding/Debugging Scenarios

- Take 3 thread dumps 10s apart; threads stuck on the same stack across dumps are truly stuck, not just sampled mid-work.

13. Best Practices

- Name threads; always join or detach; use pools with bounded queues; automate periodic thread dumps on latency alerts.

14. Practice Questions

1. Given a dump with 150 `BLOCKED` threads on `ConnectionPool`, walk through your diagnosis.
2. Map pthread lifecycle calls to Java `Thread` states.

1.8 Multithreading Models

1. Why Interviewers Ask This

Wrap-up question that checks you can connect user/kernel threads into the 1:1, N:1, M:N taxonomy and justify what real runtimes chose.

2. Core Concept

- **N:1 (many-to-one)**: all user threads on one kernel thread. Cheap, but no parallelism; one blocking call stalls all.
- **1:1 (one-to-one)**: every user thread is a kernel thread (Linux pthreads, Windows). Simple, parallel, but heavier.
- **M:N (many-to-many)**: runtime multiplexes M user threads over N kernel threads (Go GMP, Java virtual threads). Best of both, most complex.

3. Internal Working

Go's GMP: **G** (goroutine) queued on **P** (logical processor, = GOMAXPROCS) run queues, executed by **M** (kernel threads). Work stealing balances P queues; blocked M's are replaced so P's stay busy; netpoller converts socket I/O to readiness events instead of blocking M's.

4. ASCII Diagram

```
 N:1            1:1               M:N (Go GMP)
 u u u          u  u  u           G G G   G G
  \|/           |  |  |           [P run q] [P run q]
   K            K  K  K               |         |
   |            |  |  |               M1        M2   <- kernel threads
  CPU          CPU CPU CPU           CPU       CPU
one blocks all  kernel does it all  runtime + kernel cooperate
```

5. Real Production Example

- 1:1: C++/Rust services, classic Java platform threads.
- M:N: Go (Uber, Cloudflare, Kubernetes itself), Java 21+ virtual threads, Erlang/BEAM.
- N:1 pattern survives as single-threaded event loops (Node.js, Redis) — concurrency without parallelism.

6. Advantages

1:1 — simplicity + true parallelism. M:N — millions of tasks, cheap switches, still parallel.

7. Trade-offs

M:N needs a sophisticated runtime (preemption, syscall handoff, work stealing) and complicates debugging/FFI; 1:1 hits memory/switch limits at ~10^4 threads.

8. Common Mistakes

- Calling Node.js "M:N" (it's an event loop on one thread + a small worker pool).
- Assuming M:N eliminates the need to think about blocking (cgo/FFI and file I/O still pin kernel threads).

9. Performance Implications

For 1M mostly-idle connections: 1:1 needs ~1M kernel threads (infeasible); M:N or event loop handles it with N ≈ cores. CPU-bound work performs identically across models — cores are the limit.

10. Common Interview Questions

- "Compare 1:1 vs M:N; why did Linux pthreads choose 1:1?" (simplicity, kernel already fast)
- "Why did Java add virtual threads after 25 years of platform threads?"

11. Follow-up Questions

- "What is work stealing and why per-P local queues?" (cache locality, less lock contention)
- "How does GOMAXPROCS interact with container CPU limits?" (classic prod gotcha — set it to the cgroup quota)

12. Coding/Debugging Scenarios

- Go service in a 2-CPU-limit container defaults GOMAXPROCS to the host's 64 cores → heavy throttling; fix with `automaxprocs` or explicit setting.

13. Best Practices

- Choose the model per workload: event loop / M:N for massive I/O concurrency; 1:1 thread-per-core for CPU-bound; hybrid (pools) in between.

14. Practice Questions

1. Sketch how Go handles 100k goroutines each doing a blocking file read.
2. Argue for/against migrating a thread-per-request Java service to virtual threads.

Module 1 Cheat Sheet (one page)

Concept	One-liner	Key numbers/facts
Process	Isolation unit: own address space, FDs	fork = new `mm_struct`; crash contained
Thread	Execution unit: own stack/regs, shared heap	segfault kills whole process
PCB	Kernel per-task record (`task_struct`)	PID, state, regs ref, `mm`, FDs, signals
TCB	Per-thread: tid, regs, stack, sigmask, TLS	pthread stack default ~8 MB
User vs kernel threads	Who schedules: runtime vs kernel	user switch ~ns, kernel ~1–2 µs
Context switch	Save/restore task state	indirect cache/TLB cost dominates
Process states	R, S, D, T, Z on Linux	D = uninterruptible, unkillable; Z = dead PCB
Thread lifecycle	NEW→RUNNABLE→BLOCKED/ WAITING→TERMINATED	BLOCKED=lock, WAITING=condition
Models	N:1, 1:1 (pthreads), M:N (Go, vthreads)	M:N → millions of tasks

Rules of thumb: threads for sharing, processes for isolation; runnable threads ≈ cores; blocking syscalls are the enemy of green threads; load average counts D-state.

Top Interview Questions

1. Process vs thread — what's shared, what's private, crash semantics?
2. What happens during a context switch, and what's the real cost?
3. Why can Go run 1M goroutines but pthreads can't?
4. Load average 80, CPU 10% — diagnose. (D-state)
5. What happens when a multithreaded process forks?
6. Explain Java thread-dump states and how you'd triage a frozen service.
7. 1:1 vs M:N — trade-offs and real examples.

Common Mistakes (module-wide)

- "Threads are lightweight processes, end of answer" — no memory-layout detail.
- Ignoring indirect (cache/TLB) context-switch cost.
- kill -9 as a universal fix; misreading load average.
- Unbounded thread creation instead of pools.
- Forgetting fork+threads and GOMAXPROCS-in-containers traps.

Mock Interview (self-test, ~20 min)

1. (Warm-up) Chrome uses a process per tab. Defend and attack that decision.
2. (Depth) Walk through, register by register, what happens when the timer interrupt preempts thread T1 of process P1 and schedules thread T2 of process P2.
3. (Design) You must handle 500k concurrent mostly-idle websockets on 16 cores. Pick a threading model, justify memory and switch-cost math.
4. (Debugging) A Java API's p99 jumped 10×. A thread dump shows 300 threads `BLOCKED` on one monitor. What do you do next, step by step?

5. (Trap) After `fork()` your child process hangs inside `malloc`. Why?